

ISP Order Management System - Technical Specification Document

Document Information

- **Document Type:** Technical Specification
- **Version:** 1.0
- **Date:** September 2025
- **Target Audience:** Development Team, DevOps Engineers, System Architects
- **Platform:** Replit Development Environment

1. Executive Summary

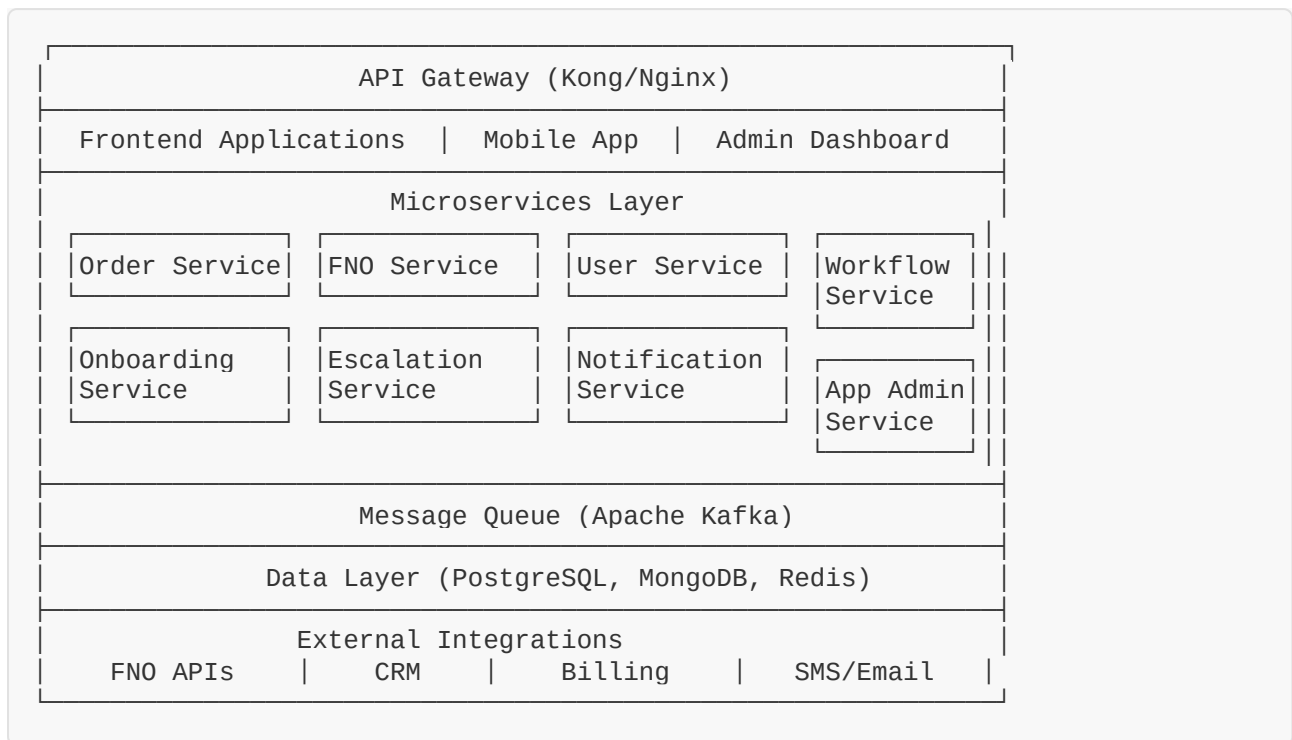
This technical specification document provides comprehensive guidelines for developing the ISP Order Management System (OMS) on the Replit platform. The system encompasses order lifecycle management, customer onboarding, FNO integration (API and manual), escalation workflows, and comprehensive system administration capabilities.

2. System Architecture Overview

2.1. Architecture Pattern

- **Pattern:** Microservices Architecture with Event-Driven Communication
- **Deployment:** Cloud-native containerized deployment
- **Communication:** RESTful APIs with message queue integration
- **Data Storage:** Multi-database approach (PostgreSQL, MongoDB, Redis)

2.2. High-Level System Components



3. Technology Stack

3.1. Backend Technologies

- **Runtime:** Node.js 18+ with TypeScript
- **Framework:** Express.js with Helmet for security
- **API Documentation:** OpenAPI 3.0 with Swagger UI
- **Authentication:** JWT with refresh tokens
- **Authorization:** Role-Based Access Control (RBAC)
- **Validation:** Joi for request validation
- **Logging:** Winston with structured logging
- **Testing:** Jest for unit tests, Supertest for integration tests

3.2. Frontend Technologies

- **Framework:** React 18+ with TypeScript
- **State Management:** Redux Toolkit with RTK Query

- **UI Library:** Material-UI (MUI) v5
- **Routing:** React Router v6
- **Forms:** React Hook Form with Yup validation
- **Charts:** Chart.js or Recharts for analytics
- **Testing:** React Testing Library with Jest

3.3. Database Technologies

- **Primary Database:** PostgreSQL 14+ for transactional data
- **Document Store:** MongoDB 6+ for flexible schemas
- **Cache:** Redis 7+ for session management and caching
- **Search:** Elasticsearch for advanced search capabilities

3.4. Infrastructure and DevOps

- **Containerization:** Docker with multi-stage builds
- **Orchestration:** Docker Compose for development
- **Message Queue:** Apache Kafka for event streaming
- **Monitoring:** Prometheus with Grafana dashboards
- **Logging:** ELK Stack (Elasticsearch, Logstash, Kibana)
- **CI/CD:** GitHub Actions for automated deployment

4. Database Design

4.1. PostgreSQL Schema Design

4.1.1. Core Tables

```
-- Users and Authentication
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  first_name VARCHAR(100) NOT NULL,
  last_name VARCHAR(100) NOT NULL,
  phone VARCHAR(20),
  role_id UUID REFERENCES roles(id),
  reporting_manager_id UUID REFERENCES users(id),
  is_active BOOLEAN DEFAULT true,
  created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);

-- Roles and Permissions
CREATE TABLE roles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(50) UNIQUE NOT NULL,
  description TEXT,
  permissions JSONB NOT NULL,
  created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);

-- Customers
CREATE TABLE customers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  customer_number VARCHAR(50) UNIQUE NOT NULL,
  first_name VARCHAR(100) NOT NULL,
  last_name VARCHAR(100) NOT NULL,
  email VARCHAR(255) NOT NULL,
  phone VARCHAR(20) NOT NULL,
  address JSONB NOT NULL,
  customer_type VARCHAR(20) DEFAULT 'individual',
  is_trial BOOLEAN DEFAULT false,
  trial_start_date DATE,
  trial_end_date DATE,
  created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);

-- Orders
CREATE TABLE orders (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  order_number VARCHAR(50) UNIQUE NOT NULL,
  customer_id UUID REFERENCES customers(id) NOT NULL,
  service_type VARCHAR(50) NOT NULL,
  service_package VARCHAR(100) NOT NULL,
  installation_address JSONB NOT NULL,
  fno_id UUID REFERENCES fnos(id),
```

```

    fno_reference VARCHAR(100),
    current_state VARCHAR(50) NOT NULL DEFAULT 'created',
    priority VARCHAR(20) DEFAULT 'normal',
    created_by UUID REFERENCES users(id),
    assigned_to UUID REFERENCES users(id),
    estimated_completion DATE,
    actual_completion DATE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Order State History
CREATE TABLE order_state_history (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    order_id UUID REFERENCES orders(id) NOT NULL,
    from_state VARCHAR(50),
    to_state VARCHAR(50) NOT NULL,
    changed_by UUID REFERENCES users(id),
    change_reason TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- FNO Configuration
CREATE TABLE fnos (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(100) NOT NULL,
    code VARCHAR(20) UNIQUE NOT NULL,
    integration_type VARCHAR(20) NOT NULL, -- 'api' or 'manual'
    api_endpoint VARCHAR(255),
    api_key_encrypted TEXT,
    portal_url VARCHAR(255),
    coverage_areas JSONB,
    is_active BOOLEAN DEFAULT true,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Application Administrator Inbox
CREATE TABLE application_inbox (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    order_id UUID REFERENCES orders(id) NOT NULL,
    fno_id UUID REFERENCES fnos(id) NOT NULL,
    assigned_to UUID REFERENCES users(id),
    priority VARCHAR(20) DEFAULT 'normal',
    status VARCHAR(20) DEFAULT 'pending',
    due_date TIMESTAMP,
    completed_at TIMESTAMP,
    notes TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Escalations
CREATE TABLE escalations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    order_id UUID REFERENCES orders(id),
    task_id UUID,
    escalation_level INTEGER DEFAULT 1,
    escalated_from UUID REFERENCES users(id),
    escalated_to UUID REFERENCES users(id),
    escalation_reason TEXT NOT NULL,
    status VARCHAR(20) DEFAULT 'open',
    resolved_at TIMESTAMP,
    resolution_notes TEXT,

```

```

        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );

-- Customer Onboarding
CREATE TABLE customer_onboarding (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    customer_id UUID REFERENCES customers(id) NOT NULL,
    order_id UUID REFERENCES orders(id),
    onboarding_type VARCHAR(20) NOT NULL, -- 'new_customer' or 'trial'
    current_step VARCHAR(50) NOT NULL,
    completion_percentage INTEGER DEFAULT 0,
    assigned_to UUID REFERENCES users(id),
    started_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    completed_at TIMESTAMP,
    notes TEXT
);

-- System Configuration
CREATE TABLE system_config (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    config_key VARCHAR(100) UNIQUE NOT NULL,
    config_value JSONB NOT NULL,
    description TEXT,
    updated_by UUID REFERENCES users(id),
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Audit Logs
CREATE TABLE audit_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id),
    action VARCHAR(100) NOT NULL,
    resource_type VARCHAR(50) NOT NULL,
    resource_id UUID,
    old_values JSONB,
    new_values JSONB,
    ip_address INET,
    user_agent TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

4.1.2. Indexes for Performance

```

-- Performance indexes
CREATE INDEX idx_orders_customer_id ON orders(customer_id);
CREATE INDEX idx_orders_state ON orders(current_state);
CREATE INDEX idx_orders_created_at ON orders(created_at);
CREATE INDEX idx_order_state_history_order_id ON order_state_history(order_id);
CREATE INDEX idx_application_inbox_assigned_to ON
application_inbox(assigned_to);
CREATE INDEX idx_application_inbox_status ON application_inbox(status);
CREATE INDEX idx_escalations_escalated_to ON escalations(escalated_to);
CREATE INDEX idx_audit_logs_user_id ON audit_logs(user_id);
CREATE INDEX idx_audit_logs_created_at ON audit_logs(created_at);

```

4.2. MongoDB Collections

4.2.1. Communication Logs

```
// communications collection
{
  _id: ObjectId,
  orderId: String,
  customerId: String,
  type: String, // 'email', 'sms', 'system_notification'
  direction: String, // 'inbound', 'outbound'
  subject: String,
  content: String,
  templateId: String,
  status: String, // 'sent', 'delivered', 'failed'
  metadata: {
    fnoId: String,
    campaignId: String,
    retryCount: Number
  },
  createdAt: Date,
  deliveredAt: Date
}
```

4.2.2. FNO Integration Logs

```
// fno_integration_logs collection
{
  _id: ObjectId,
  orderId: String,
  fnoId: String,
  integrationType: String, // 'api', 'manual'
  action: String, // 'submit', 'status_update', 'query'
  request: {
    method: String,
    url: String,
    headers: Object,
    body: Object
  },
  response: {
    statusCode: Number,
    headers: Object,
    body: Object
  },
  processingTime: Number,
  success: Boolean,
  errorMessage: String,
  createdAt: Date
}
```

4.2.3. Workflow Definitions

```
// workflow_definitions collection
{
  _id: ObjectId,
  name: String,
  version: String,
  orderType: String,
  serviceType: String,
  steps: [{
    id: String,
    name: String,
    type: String, // 'manual', 'automated', 'approval'
    assigneeRole: String,
    slaHours: Number,
    escalationRules: [{
      level: Number,
      afterHours: Number,
      escalateTo: String // 'reporting_manager', 'process_owner'
    }],
    conditions: Object,
    actions: Object
  }],
  isActive: Boolean,
  createdBy: String,
  createdAt: Date,
  updatedAt: Date
}
```

4.3. Redis Cache Structure

```
// Session management
"session:{sessionId}" = {
  userId: String,
  email: String,
  role: String,
  permissions: Array,
  lastActivity: Number,
  expiresAt: Number
}

// Order cache for quick access
"order:{orderId}" = {
  orderNumber: String,
  customerId: String,
  currentState: String,
  priority: String,
  assignedTo: String,
  lastUpdated: Number
}

// Application inbox cache
"inbox:{userId}" = {
  pendingCount: Number,
  highPriorityCount: Number,
  overdueCount: Number,
  lastRefresh: Number
}
```

5. API Specifications

5.1. Authentication APIs

5.1.1. User Authentication

```
// POST /api/auth/login
interface LoginRequest {
  email: string;
  password: string;
}

interface LoginResponse {
  success: boolean;
  data: {
    user: {
      id: string;
      email: string;
      firstName: string;
      lastName: string;
      role: string;
      permissions: string[];
    };
    accessToken: string;
    refreshToken: string;
    expiresIn: number;
  };
}

// POST /api/auth/refresh
interface RefreshTokenRequest {
  refreshToken: string;
}

// POST /api/auth/logout
interface LogoutRequest {
  refreshToken: string;
}
```

5.2. Order Management APIs

5.2.1. Order CRUD Operations

```
// POST /api/orders
interface CreateOrderRequest {
  customerId: string;
  serviceType: string;
  servicePackage: string;
  installationAddress: {
    street: string;
    city: string;
    province: string;
    postalCode: string;
    coordinates?: {
      latitude: number;
      longitude: number;
    };
  };
  priority?: 'low' | 'normal' | 'high' | 'urgent';
  notes?: string;
}

interface CreateOrderResponse {
  success: boolean;
  data: {
    orderId: string;
    orderNumber: string;
    currentState: string;
    estimatedCompletion: string;
  };
}

// GET /api/orders/{orderId}
interface GetOrderResponse {
  success: boolean;
  data: {
    id: string;
    orderNumber: string;
    customer: CustomerInfo;
    serviceType: string;
    servicePackage: string;
    currentState: string;
    stateHistory: OrderStateHistory[];
    fno: FNOInfo;
    fnoReference?: string;
    priority: string;
    assignedTo?: UserInfo;
    createdAt: string;
    updatedAt: string;
    estimatedCompletion?: string;
    actualCompletion?: string;
  };
}

// PUT /api/orders/{orderId}/state
interface UpdateOrderStateRequest {
  newState: string;
  reason?: string;
}
```

```
    notes?: string;
}

// GET /api/orders
interface GetOrdersRequest {
    page?: number;
    limit?: number;
    state?: string;
    customerId?: string;
    assignedTo?: string;
    priority?: string;
    dateFrom?: string;
    dateTo?: string;
    search?: string;
}
```

5.3. FNO Integration APIs

5.3.1. FNO Management

```
// GET /api/fnos
interface GetFNOSResponse {
  success: boolean;
  data: {
    fnos: FNOInfo[];
    total: number;
  };
}

interface FNOInfo {
  id: string;
  name: string;
  code: string;
  integrationType: 'api' | 'manual';
  apiEndpoint?: string;
  portalUrl?: string;
  coverageAreas: string[];
  isActive: boolean;
}

// POST /api/fnos/{fnoId}/submit-order
interface SubmitOrderToFNORequest {
  orderId: string;
  submissionType: 'api' | 'manual';
}

interface SubmitOrderToFNOResponse {
  success: boolean;
  data: {
    submissionId: string;
    fnoReference?: string;
    status: string;
    message?: string;
  };
}

// PUT /api/fnos/manual-application/{applicationId}
interface UpdateManualApplicationRequest {
  fnoReference: string;
  status: 'submitted' | 'accepted' | 'rejected' | 'in_progress';
  notes?: string;
}
```

5.4. Application Administrator APIs

5.4.1. Inbox Management

```
// GET /api/application-admin/inbox
interface GetInboxRequest {
  page?: number;
  limit?: number;
  status?: 'pending' | 'in_progress' | 'completed';
  priority?: 'low' | 'normal' | 'high' | 'urgent';
  fnoId?: string;
  overdue?: boolean;
}

interface GetInboxResponse {
  success: boolean;
  data: {
    applications: ApplicationInboxItem[];
    total: number;
    summary: {
      pending: number;
      inProgress: number;
      overdue: number;
      highPriority: number;
    };
  };
}

interface ApplicationInboxItem {
  id: string;
  order: {
    id: string;
    orderNumber: string;
    customer: CustomerInfo;
    serviceType: string;
  };
  fno: FNOInfo;
  priority: string;
  status: string;
  dueDate: string;
  assignedTo?: UserInfo;
  createdAt: string;
  agingHours: number;
}

// PUT /api/application-admin/inbox/{applicationId}/assign
interface AssignApplicationRequest {
  assignedTo: string;
}

// PUT /api/application-admin/inbox/{applicationId}/complete
interface CompleteApplicationRequest {
  fnoReference: string;
  notes?: string;
}
```

5.5. Escalation Management APIs

5.5.1. Escalation Operations

```
// POST /api/escalations
interface CreateEscalationRequest {
  orderId?: string;
  taskId?: string;
  escalationReason: string;
  escalationLevel: number;
  escalatedTo: string;
  priority: 'low' | 'normal' | 'high' | 'urgent';
}

// GET /api/escalations/my-escalations
interface GetMyEscalationsResponse {
  success: boolean;
  data: {
    escalations: EscalationInfo[];
    total: number;
    summary: {
      open: number;
      overdue: number;
      resolved: number;
    };
  };
}

interface EscalationInfo {
  id: string;
  order?: OrderInfo;
  escalationLevel: number;
  escalatedFrom: UserInfo;
  escalationReason: string;
  status: string;
  createdAt: string;
  dueDate?: string;
  agingHours: number;
}

// PUT /api/escalations/{escalationId}/resolve
interface ResolveEscalationRequest {
  resolutionNotes: string;
  escalateToNextLevel?: boolean;
}
```

5.6. Customer Onboarding APIs

5.6.1. Onboarding Management

```
// POST /api/onboarding/initiate
interface InitiateOnboardingRequest {
  customerId: string;
  orderId?: string;
  onboardingType: 'new_customer' | 'trial';
  assignedTo?: string;
}

// GET /api/onboarding/customers/{customerId}
interface GetCustomerOnboardingResponse {
  success: boolean;
  data: {
    id: string;
    customer: CustomerInfo;
    onboardingType: string;
    currentStep: string;
    completionPercentage: number;
    steps: OnboardingStep[];
    assignedTo?: UserInfo;
    startedAt: string;
    completedAt?: string;
  };
}

interface OnboardingStep {
  id: string;
  name: string;
  description: string;
  status: 'pending' | 'in_progress' | 'completed' | 'skipped';
  completedAt?: string;
  completedBy?: UserInfo;
  notes?: string;
}

// PUT /api/onboarding/{onboardingId}/step/{stepId}/complete
interface CompleteOnboardingStepRequest {
  notes?: string;
}

// GET /api/onboarding/trial-customers
interface GetTrialCustomersResponse {
  success: boolean;
  data: {
    customers: TrialCustomerInfo[];
    total: number;
    summary: {
      active: number;
      expiringSoon: number; // within 7 days
      converted: number;
      expired: number;
    };
  };
}

interface TrialCustomerInfo {
```

```
id: string;
customerNumber: string;
name: string;
email: string;
trialStartDate: string;
trialEndDate: string;
daysRemaining: number;
engagementScore: number;
lastActivity: string;
conversionCampaigns: number;
}
```

5.7. System Administration APIs

5.7.1. User Management

```
// POST /api/admin/users
interface CreateUserRequest {
  email: string;
  firstName: string;
  lastName: string;
  phone?: string;
  roleId: string;
  reportingManagerId?: string;
  temporaryPassword?: string;
}

// GET /api/admin/users
interface GetUsersRequest {
  page?: number;
  limit?: number;
  roleId?: string;
  isActive?: boolean;
  search?: string;
}

// PUT /api/admin/users/{userId}
interface UpdateUserRequest {
  firstName?: string;
  lastName?: string;
  phone?: string;
  roleId?: string;
  reportingManagerId?: string;
  isActive?: boolean;
}

// PUT /api/admin/users/{userId}/reset-password
interface ResetPasswordRequest {
  temporaryPassword: string;
  forcePasswordChange: boolean;
}
```

5.7.2. Role and Permission Management

```
// POST /api/admin/roles
interface CreateRoleRequest {
  name: string;
  description?: string;
  permissions: string[];
}

// GET /api/admin/roles
interface GetRolesResponse {
  success: boolean;
  data: {
    roles: RoleInfo[];
    availablePermissions: PermissionInfo[];
  };
}

interface RoleInfo {
  id: string;
  name: string;
  description?: string;
  permissions: string[];
  userCount: number;
  createdAt: string;
}

interface PermissionInfo {
  id: string;
  name: string;
  description: string;
  category: string;
}
```

5.7.3. System Configuration

```
// GET /api/admin/config
interface GetSystemConfigResponse {
  success: boolean;
  data: {
    configs: SystemConfigItem[];
  };
}

interface SystemConfigItem {
  id: string;
  configKey: string;
  configValue: any;
  description?: string;
  updatedBy?: UserInfo;
  updatedAt: string;
}

// PUT /api/admin/config/{configKey}
interface UpdateSystemConfigRequest {
  configValue: any;
}

// Common configuration keys:
// - escalation.default_sla_hours
// - escalation.reporting_manager_escalation_hours
// - escalation.process_owner_escalation_hours
// - trial.default_duration_days
// - trial.reminder_days
// - onboarding.default_steps
// - fno.api_timeout_seconds
// - notifications.email_templates
```

6. Security Requirements

6.1. Authentication and Authorization

6.1.1. JWT Token Implementation

```
interface JWTPayload {
    userId: string;
    email: string;
    role: string;
    permissions: string[];
    iat: number;
    exp: number;
}

// Token configuration
const JWT_CONFIG = {
    accessTokenExpiry: '15m',
    refreshTokenExpiry: '7d',
    algorithm: 'HS256',
    issuer: 'isp-oms',
    audience: 'isp-oms-users'
};
```

6.1.2. Permission System

```
// Permission categories and specific permissions
const PERMISSIONS = {
  ORDERS: {
    CREATE: 'orders:create',
    READ: 'orders:read',
    UPDATE: 'orders:update',
    DELETE: 'orders:delete',
    ASSIGN: 'orders:assign',
    ESCALATE: 'orders:escalate'
  },
  CUSTOMERS: {
    CREATE: 'customers:create',
    READ: 'customers:read',
    UPDATE: 'customers:update',
    DELETE: 'customers:delete'
  },
  FNO: {
    CONFIGURE: 'fno:configure',
    SUBMIT_API: 'fno:submit_api',
    SUBMIT_MANUAL: 'fno:submit_manual',
    VIEW_LOGS: 'fno:view_logs'
  },
  APPLICATION_ADMIN: {
    VIEW_INBOX: 'app_admin:view_inbox',
    PROCESS_APPLICATIONS: 'app_admin:process_applications',
    ASSIGN_APPLICATIONS: 'app_admin:assign_applications'
  },
  ESCALATIONS: {
    VIEW: 'escalations:view',
    RESOLVE: 'escalations:resolve',
    ESCALATE: 'escalations:escalate'
  },
  ONBOARDING: {
    INITIATE: 'onboarding:initiate',
    MANAGE: 'onboarding:manage',
    VIEW_TRIALS: 'onboarding:view_trials',
    MANAGE_CAMPAIGNS: 'onboarding:manage_campaigns'
  },
  SYSTEM_ADMIN: {
    MANAGE_USERS: 'admin:manage_users',
    MANAGE_ROLES: 'admin:manage_roles',
    SYSTEM_CONFIG: 'admin:system_config',
    VIEW_AUDIT_LOGS: 'admin:view_audit_logs',
    SYSTEM_MONITORING: 'admin:system_monitoring'
  }
};

// Role definitions
const ROLES = {
  SUPER_ADMIN: {
    name: 'Super Administrator',
    permissions: Object.values(PERMISSIONS).flatMap(category =>
Object.values(category))
  },
  SYSTEM_ADMIN: {
    name: 'System Administrator',
    permissions: [
      ...Object.values(PERMISSIONS.SYSTEM_ADMIN),
      PERMISSIONS.ORDERS.READ,

```

```

        PERMISSIONS.CUSTOMERS.READ
    ]
},
OPERATIONS_MANAGER: {
    name: 'Operations Manager',
    permissions: [
        ...Object.values(PERMISSIONS.ORDERS),
        ...Object.values(PERMISSIONS.ESCALATIONS),
        PERMISSIONS.CUSTOMERS.READ,
        PERMISSIONS.FNO.VIEW_LOGS
    ]
},
APPLICATION_ADMIN: {
    name: 'Application Administrator',
    permissions: [
        ...Object.values(PERMISSIONS.APPLICATION_ADMIN),
        PERMISSIONS.ORDERS.READ,
        PERMISSIONS.ORDERS.UPDATE,
        PERMISSIONS.FNO.SUBMIT_MANUAL
    ]
},
CUSTOMER_SUCCESS_MANAGER: {
    name: 'Customer Success Manager',
    permissions: [
        ...Object.values(PERMISSIONS.ONBOARDING),
        PERMISSIONS.CUSTOMERS.READ,
        PERMISSIONS.CUSTOMERS.UPDATE,
        PERMISSIONS.ORDERS.READ
    ]
},
SALES_REPRESENTATIVE: {
    name: 'Sales Representative',
    permissions: [
        PERMISSIONS.ORDERS.CREATE,
        PERMISSIONS.ORDERS.READ,
        PERMISSIONS.ORDERS.UPDATE,
        PERMISSIONS.CUSTOMERS.CREATE,
        PERMISSIONS.CUSTOMERS.READ,
        PERMISSIONS.CUSTOMERS.UPDATE
    ]
}
};

```

6.2. Data Security

6.2.1. Encryption Requirements

```
// Sensitive data encryption
interface EncryptionConfig {
  algorithm: 'aes-256-gcm';
  keyDerivation: 'pbkdf2';
  iterations: 100000;
  saltLength: 32;
  ivLength: 16;
  tagLength: 16;
}

// Fields requiring encryption
const ENCRYPTED_FIELDS = [
  'users.password_hash',
  'fnos.api_key_encrypted',
  'customers.personal_data', // PII fields
  'audit_logs.sensitive_data'
];
```

6.2.2. Input Validation and Sanitization

```
// Validation schemas using Joi
const orderValidationSchema = Joi.object({
  customerId: Joi.string().uuid().required(),
  serviceType: Joi.string().valid('fiber', 'wireless', 'hybrid').required(),
  servicePackage: Joi.string().min(1).max(100).required(),
  installationAddress: Joi.object({
    street: Joi.string().min(1).max(200).required(),
    city: Joi.string().min(1).max(100).required(),
    province: Joi.string().min(1).max(100).required(),
    postalCode: Joi.string().pattern(/^[0-9]{4}$/).required(),
    coordinates: Joi.object({
      latitude: Joi.number().min(-90).max(90),
      longitude: Joi.number().min(-180).max(180)
    }).optional()
  }).required(),
  priority: Joi.string().valid('low', 'normal', 'high', 'urgent').default('normal'),
  notes: Joi.string().max(1000).optional()
});
```

7. Development Guidelines

7.1. Project Structure

```
isp-oms/
├── backend/
│   ├── src/
│   │   ├── controllers/
│   │   │   ├── auth.controller.ts
│   │   │   ├── orders.controller.ts
│   │   │   ├── fno.controller.ts
│   │   │   ├── application-admin.controller.ts
│   │   │   ├── escalations.controller.ts
│   │   │   ├── onboarding.controller.ts
│   │   │   └── admin.controller.ts
│   │   ├── services/
│   │   │   ├── auth.service.ts
│   │   │   ├── orders.service.ts
│   │   │   ├── fno-integration.service.ts
│   │   │   ├── workflow.service.ts
│   │   │   ├── escalation.service.ts
│   │   │   ├── notification.service.ts
│   │   │   └── audit.service.ts
│   │   ├── models/
│   │   │   ├── user.model.ts
│   │   │   ├── order.model.ts
│   │   │   ├── customer.model.ts
│   │   │   └── fno.model.ts
│   │   ├── middleware/
│   │   │   ├── auth.middleware.ts
│   │   │   ├── validation.middleware.ts
│   │   │   ├── audit.middleware.ts
│   │   │   └── error.middleware.ts
│   │   ├── utils/
│   │   │   ├── encryption.util.ts
│   │   │   ├── logger.util.ts
│   │   │   ├── email.util.ts
│   │   │   └── sms.util.ts
│   │   ├── config/
│   │   │   ├── database.config.ts
│   │   │   ├── redis.config.ts
│   │   │   ├── kafka.config.ts
│   │   │   └── app.config.ts
│   │   ├── migrations/
│   │   ├── seeds/
│   │   └── tests/
│   ├── package.json
│   ├── tsconfig.json
│   ├── jest.config.js
│   └── Dockerfile
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   │   ├── common/
│   │   │   ├── orders/
│   │   │   ├── customers/
│   │   │   ├── application-admin/
│   │   │   └── escalations/
```

```

├── onboarding/
├── admin/
├── pages/
├── hooks/
├── services/
├── store/
├── utils/
├── types/
├── public/
├── package.json
├── tsconfig.json
├── Dockerfile
├── docker-compose.yml
├── docker-compose.prod.yml
├── .github/
│   └── workflows/
│       ├── ci.yml
│       └── cd.yml
└── README.md

```

7.2. Coding Standards

7.2.1. TypeScript Configuration

```

// tsconfig.json
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "lib": ["ES2020"],
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "resolveJsonModule": true,
    "declaration": true,
    "declarationMap": true,
    "sourceMap": true,
    "experimentalDecorators": true,
    "emitDecoratorMetadata": true,
    "baseUrl": "./src",
    "paths": {
      "@controllers/*": ["controllers/*"],
      "@services/*": ["services/*"],
      "@models/*": ["models/*"],
      "@utils/*": ["utils/*"],
      "@config/*": ["config/*"]
    }
  },
  "include": ["src/**/*.ts"],
  "exclude": ["node_modules", "dist", "tests"]
}

```

7.2.2. ESLint Configuration

```
// .eslintrc.json
{
  "extends": [
    "@typescript-eslint/recommended",
    "prettier"
  ],
  "parser": "@typescript-eslint/parser",
  "plugins": ["@typescript-eslint"],
  "rules": {
    "@typescript-eslint/no-unused-vars": "error",
    "@typescript-eslint/explicit-function-return-type": "warn",
    "@typescript-eslint/no-explicit-any": "warn",
    "prefer-const": "error",
    "no-var": "error"
  }
}
```

7.3. Error Handling

7.3.1. Custom Error Classes

```
// Base error class
export class AppError extends Error {
  public readonly statusCode: number;
  public readonly isOperational: boolean;
  public readonly errorCode: string;

  constructor(
    message: string,
    statusCode: number = 500,
    errorCode: string = 'INTERNAL_ERROR',
    isOperational: boolean = true
  ) {
    super(message);
    this.statusCode = statusCode;
    this.errorCode = errorCode;
    this.isOperational = isOperational;

    Error.captureStackTrace(this, this.constructor);
  }
}

// Specific error classes
export class ValidationError extends AppError {
  constructor(message: string, field?: string) {
    super(message, 400, 'VALIDATION_ERROR');
    this.name = 'ValidationError';
  }
}

export class AuthenticationError extends AppError {
  constructor(message: string = 'Authentication failed') {
    super(message, 401, 'AUTHENTICATION_ERROR');
    this.name = 'AuthenticationError';
  }
}

export class AuthorizationError extends AppError {
  constructor(message: string = 'Insufficient permissions') {
    super(message, 403, 'AUTHORIZATION_ERROR');
    this.name = 'AuthorizationError';
  }
}

export class NotFoundError extends AppError {
  constructor(resource: string) {
    super(`${resource} not found`, 404, 'NOT_FOUND');
    this.name = 'NotFoundError';
  }
}

export class FNOIntegrationError extends AppError {
  constructor(message: string, fnoId?: string) {
    super(message, 502, 'FNO_INTEGRATION_ERROR');
    this.name = 'FNOIntegrationError';
  }
}
```

```
}  
}
```

7.3.2. Global Error Handler

```
// Error handling middleware  
export const errorHandler = (  
  error: Error,  
  req: Request,  
  res: Response,  
  next: NextFunction  
): void => {  
  let appError = error as AppError;  
  
  // Convert known errors to AppError  
  if (!(error instanceof AppError)) {  
    if (error.name === 'ValidationError') {  
      appError = new ValidationError(error.message);  
    } else if (error.name === 'CastError') {  
      appError = new ValidationError('Invalid ID format');  
    } else {  
      appError = new AppError('Internal server error', 500, 'INTERNAL_ERROR',  
false);  
    }  
  }  
  
  // Log error  
  logger.error({  
    error: {  
      message: appError.message,  
      stack: appError.stack,  
      statusCode: appError.statusCode,  
      errorCode: appError.errorCode  
    },  
    request: {  
      method: req.method,  
      url: req.url,  
      userId: req.user?.id,  
      ip: req.ip  
    }  
  });  
  
  // Send error response  
  res.status(appError.statusCode).json({  
    success: false,  
    error: {  
      message: appError.message,  
      code: appError.errorCode,  
      ...(process.env.NODE_ENV === 'development' && { stack: appError.stack })  
    }  
  });  
};
```

8. Testing Strategy

8.1. Unit Testing

8.1.1. Service Layer Testing

```
// Example: orders.service.test.ts
import { OrdersService } from '@services/orders.service';
import { OrderRepository } from '@repositories/order.repository';
import { FNOService } from '@services/fno.service';
import { NotificationService } from '@services/notification.service';

describe('OrdersService', () => {
  let ordersService: OrdersService;
  let mockOrderRepository: jest.Mocked<OrderRepository>;
  let mockFNOService: jest.Mocked<FNOService>;
  let mockNotificationService: jest.Mocked<NotificationService>;

  beforeEach(() => {
    mockOrderRepository = {
      create: jest.fn(),
      findById: jest.fn(),
      update: jest.fn(),
      findByCustomerId: jest.fn()
    } as any;

    mockFNOService = {
      submitOrder: jest.fn(),
      getFNOForLocation: jest.fn()
    } as any;

    mockNotificationService = {
      sendOrderConfirmation: jest.fn()
    } as any;

    ordersService = new OrdersService(
      mockOrderRepository,
      mockFNOService,
      mockNotificationService
    );
  });

  describe('createOrder', () => {
    it('should create order successfully', async () => {
      // Arrange
      const orderData = {
        customerId: 'customer-123',
        serviceType: 'fiber',
        servicePackage: 'premium',
        installationAddress: {
          street: '123 Main St',
          city: 'Cape Town',
          province: 'Western Cape',
          postalCode: '8001'
        }
      };
    });
  });
});
```

```

    const mockOrder = {
      id: 'order-123',
      orderNumber: 'ORD-2025-001',
      ...orderData,
      currentState: 'created'
    };

    mockOrderRepository.create.mockResolvedValue(mockOrder);
    mockFNOService.getFNOForLocation.mockResolvedValue({
      id: 'fno-123',
      name: 'Test FNO',
      integrationType: 'api'
    });

    // Act
    const result = await ordersService.createOrder(orderData);

    // Assert
    expect(result).toEqual(mockOrder);
    expect(mockOrderRepository.create).toHaveBeenCalledWith(
      expect.objectContaining(orderData)
    );

    expect(mockNotificationService.sendOrderConfirmation).toHaveBeenCalledWith(
      mockOrder
    );
  });

  it('should throw ValidationError for invalid address', async () => {
    // Arrange
    const invalidOrderData = {
      customerId: 'customer-123',
      serviceType: 'fiber',
      servicePackage: 'premium',
      installationAddress: {
        street: '',
        city: 'Cape Town',
        province: 'Western Cape',
        postalCode: 'invalid'
      }
    };

    // Act & Assert
    await expect(ordersService.createOrder(invalidOrderData))
      .rejects
      .toThrow(ValidationError);
  });
});
});
});

```

8.2. Integration Testing

8.2.1. API Integration Tests

```
// Example: orders.integration.test.ts
import request from 'supertest';
import { app } from '../src/app';
import { setupTestDatabase, cleanupTestDatabase } from '../helpers/database';
import { createTestUser, getAuthToken } from '../helpers/auth';

describe('Orders API Integration', () => {
  let authToken: string;
  let testUser: any;

  beforeAll(async () => {
    await setupTestDatabase();
    testUser = await createTestUser({
      email: 'test@example.com',
      role: 'sales_representative'
    });
    authToken = await getAuthToken(testUser);
  });

  afterAll(async () => {
    await cleanupTestDatabase();
  });

  describe('POST /api/orders', () => {
    it('should create order successfully', async () => {
      const orderData = {
        customerId: 'customer-123',
        serviceType: 'fiber',
        servicePackage: 'premium',
        installationAddress: {
          street: '123 Main St',
          city: 'Cape Town',
          province: 'Western Cape',
          postalCode: '8001'
        }
      };

      const response = await request(app)
        .post('/api/orders')
        .set('Authorization', `Bearer ${authToken}`)
        .send(orderData)
        .expect(201);

      expect(response.body.success).toBe(true);
      expect(response.body.data).toHaveProperty('orderId');
      expect(response.body.data).toHaveProperty('orderNumber');
      expect(response.body.data.currentState).toBe('created');
    });

    it('should return 401 for unauthenticated request', async () => {
      const orderData = {
        customerId: 'customer-123',
        serviceType: 'fiber',
        servicePackage: 'premium'
      };
    });
  });
});
```

```
    await request(app)
      .post('/api/orders')
      .send(orderData)
      .expect(401);
  });
});
});
```

8.3. End-to-End Testing

8.3.1. Cypress E2E Tests

```
// cypress/integration/order-management.spec.ts
describe('Order Management Flow', () => {
  beforeEach(() => {
    cy.login('sales@example.com', 'password');
  });

  it('should complete full order creation flow', () => {
    // Navigate to create order page
    cy.visit('/orders/create');

    // Fill customer information
    cy.get('[data-testid=customer-search]').type('John Doe');
    cy.get('[data-testid=customer-option]').first().click();

    // Select service type
    cy.get('[data-testid=service-type]').select('fiber');
    cy.get('[data-testid=service-package]').select('premium');

    // Fill installation address
    cy.get('[data-testid=street-address]').type('123 Main Street');
    cy.get('[data-testid=city]').type('Cape Town');
    cy.get('[data-testid=province]').select('Western Cape');
    cy.get('[data-testid=postal-code]').type('8001');

    // Submit order
    cy.get('[data-testid=submit-order]').click();

    // Verify success
    cy.get('[data-testid=success-message]').should('be.visible');
    cy.get('[data-testid=order-number]').should('contain', 'ORD-');

    // Verify order appears in orders list
    cy.visit('/orders');
    cy.get('[data-testid=orders-table]').should('contain', 'John Doe');
  });

  it('should handle FNO manual application workflow', () => {
    // Create order that requires manual FNO application
    cy.createOrder({
      customer: 'Jane Smith',
      serviceType: 'fiber',
      location: 'Manual FNO Area'
    });

    // Login as Application Administrator
    cy.login('appadmin@example.com', 'password');

    // Check inbox
    cy.visit('/application-admin/inbox');
    cy.get('[data-testid=pending-applications]').should('contain', 'Jane Smith');

    // Process application
    cy.get('[data-testid=application-row]').first().click();
    cy.get('[data-testid=fno-reference]').type('FNO-REF-12345');
```

```
cy.get('[data-testid=complete-application]').click();

// Verify completion
cy.get('[data-testid=success-message]').should('contain', 'Application
completed');
});
});
```

9. Deployment and Infrastructure

9.1. Docker Configuration

9.1.1. Backend Dockerfile

```
# Backend Dockerfile
FROM node:18-alpine AS builder

WORKDIR /app

# Copy package files
COPY package*.json ./
COPY tsconfig.json ./

# Install dependencies
RUN npm ci --only=production

# Copy source code
COPY src/ ./src/

# Build application
RUN npm run build

# Production stage
FROM node:18-alpine AS production

WORKDIR /app

# Create non-root user
RUN addgroup -g 1001 -S nodejs
RUN adduser -S nodejs -u 1001

# Copy built application
COPY --from=builder --chown=nodejs:nodejs /app/dist ./dist
COPY --from=builder --chown=nodejs:nodejs /app/node_modules ./node_modules
COPY --from=builder --chown=nodejs:nodejs /app/package.json ./

# Switch to non-root user
USER nodejs

# Expose port
EXPOSE 3000

# Health check
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD curl -f http://localhost:3000/health || exit 1

# Start application
CMD ["node", "dist/index.js"]
```

9.1.2. Frontend Dockerfile

```
# Frontend Dockerfile
FROM node:18-alpine AS builder

WORKDIR /app

# Copy package files
COPY package*.json ./

# Install dependencies
RUN npm ci

# Copy source code
COPY . .

# Build application
RUN npm run build

# Production stage
FROM nginx:alpine AS production

# Copy built application
COPY --from=builder /app/build /usr/share/nginx/html

# Copy nginx configuration
COPY nginx.conf /etc/nginx/nginx.conf

# Expose port
EXPOSE 80

# Start nginx
CMD ["nginx", "-g", "daemon off;"]
```

9.2. Docker Compose Configuration

9.2.1. Development Environment

```
# docker-compose.yml
version: '3.8'

services:
  # PostgreSQL Database
  postgres:
    image: postgres:14-alpine
    environment:
      POSTGRES_DB: isp_oms
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./backend/migrations:/docker-entrypoint-initdb.d
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5

  # MongoDB
  mongodb:
    image: mongo:6-alpine
    environment:
      MONGO_INITDB_ROOT_USERNAME: admin
      MONGO_INITDB_ROOT_PASSWORD: admin
      MONGO_INITDB_DATABASE: isp_oms_logs
    ports:
      - "27017:27017"
    volumes:
      - mongodb_data:/data/db

  # Redis
  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data
    command: redis-server --appendonly yes

  # Apache Kafka
  zookeeper:
    image: confluentinc/cp-zookeeper:latest
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000

  kafka:
    image: confluentinc/cp-kafka:latest
    depends_on:
      - zookeeper
    ports:
```

```

- "9092:9092"
environment:
  KAFKA_BROKER_ID: 1
  KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
  KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
  KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

# Backend API
backend:
  build:
    context: ./backend
    dockerfile: Dockerfile
  ports:
    - "3000:3000"
  environment:
    NODE_ENV: development
    DATABASE_URL: postgresql://postgres:postgres@postgres:5432/isp_oms
    MONGODB_URL: mongodb://admin:admin@mongodb:27017/isp_oms_logs?
authSource=admin
    REDIS_URL: redis://redis:6379
    KAFKA_BROKERS: kafka:9092
    JWT_SECRET: your-jwt-secret-key
    JWT_REFRESH_SECRET: your-jwt-refresh-secret-key
  depends_on:
    postgres:
      condition: service_healthy
    mongodb:
      condition: service_started
    redis:
      condition: service_started
    kafka:
      condition: service_started
  volumes:
    - ./backend/src:/app/src
    - ./backend/uploads:/app/uploads
  command: npm run dev

# Frontend Application
frontend:
  build:
    context: ./frontend
    dockerfile: Dockerfile
  ports:
    - "3001:80"
  environment:
    REACT_APP_API_URL: http://localhost:3000/api
  depends_on:
    - backend

# Nginx API Gateway
nginx:
  image: nginx:alpine
  ports:
    - "80:80"
  volumes:
    - ./nginx.conf:/etc/nginx/nginx.conf
  depends_on:
    - backend
    - frontend

volumes:
  postgres_data:

```

```
mongodb_data:
```

```
redis_data:
```

9.2.2. Production Environment

```
# docker-compose.prod.yml
version: '3.8'

services:
  postgres:
    image: postgres:14-alpine
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    restart: unless-stopped
    networks:
      - backend

  mongodb:
    image: mongo:6-alpine
    environment:
      MONGO_INITDB_ROOT_USERNAME: ${MONGO_USER}
      MONGO_INITDB_ROOT_PASSWORD: ${MONGO_PASSWORD}
    volumes:
      - mongodb_data:/data/db
    restart: unless-stopped
    networks:
      - backend

  redis:
    image: redis:7-alpine
    volumes:
      - redis_data:/data
    restart: unless-stopped
    networks:
      - backend

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
      target: production
    environment:
      NODE_ENV: production
      DATABASE_URL: ${DATABASE_URL}
      MONGODB_URL: ${MONGODB_URL}
      REDIS_URL: ${REDIS_URL}
      JWT_SECRET: ${JWT_SECRET}
      JWT_REFRESH_SECRET: ${JWT_REFRESH_SECRET}
    depends_on:
      - postgres
      - mongodb
      - redis
    restart: unless-stopped
    networks:
      - backend
      - frontend
    deploy:
      replicas: 3
      resources:
        limits:
```

```
    cpus: '1'
    memory: 1G

frontend:
  build:
    context: ./frontend
    dockerfile: Dockerfile
    target: production
  environment:
    REACT_APP_API_URL: ${API_URL}
  restart: unless-stopped
  networks:
    - frontend
  deploy:
    replicas: 2

nginx:
  image: nginx:alpine
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./nginx.prod.conf:/etc/nginx/nginx.conf
    - ./ssl:/etc/nginx/ssl
  depends_on:
    - backend
    - frontend
  restart: unless-stopped
  networks:
    - frontend

networks:
  backend:
    driver: bridge
  frontend:
    driver: bridge

volumes:
  postgres_data:
  mongodb_data:
  redis_data:
```

9.3. Environment Configuration

9.3.1. Environment Variables

```
# .env.example
# Database Configuration
DATABASE_URL=postgresql://username:password@localhost:5432/isp_oms
MONGODB_URL=mongodb://username:password@localhost:27017/isp_oms_logs
REDIS_URL=redis://localhost:6379

# JWT Configuration
JWT_SECRET=your-super-secret-jwt-key-change-in-production
JWT_REFRESH_SECRET=your-super-secret-refresh-key-change-in-production
JWT_EXPIRES_IN=15m
JWT_REFRESH_EXPIRES_IN=7d

# Application Configuration
NODE_ENV=development
PORT=3000
API_PREFIX=/api
CORS_ORIGIN=http://localhost:3001

# External Service Configuration
# Email Service (SendGrid, AWS SES, etc.)
EMAIL_SERVICE=sendgrid
EMAIL_API_KEY=your-email-service-api-key
EMAIL_FROM=noreply@yourcompany.com

# SMS Service (Twilio, AWS SNS, etc.)
SMS_SERVICE=twilio
SMS_ACCOUNT_SID=your-twilio-account-sid
SMS_AUTH_TOKEN=your-twilio-auth-token
SMS_FROM=+1234567890

# File Upload Configuration
UPLOAD_MAX_SIZE=10485760 # 10MB
UPLOAD_ALLOWED_TYPES=image/jpeg,image/png,application/pdf

# Logging Configuration
LOG_LEVEL=info
LOG_FORMAT=json

# Monitoring Configuration
PROMETHEUS_ENABLED=true
PROMETHEUS_PORT=9090

# Rate Limiting
RATE_LIMIT_WINDOW_MS=900000 # 15 minutes
RATE_LIMIT_MAX_REQUESTS=100

# Session Configuration
SESSION_SECRET=your-session-secret-key
SESSION_TIMEOUT=3600000 # 1 hour

# FNO Integration Configuration
FNO_API_TIMEOUT=30000 # 30 seconds
FNO_RETRY_ATTEMPTS=3
FNO_RETRY_DELAY=1000 # 1 second
```

```
# Escalation Configuration
ESCALATION_CHECK_INTERVAL=300000 # 5 minutes
DEFAULT_SLA_HOURS=24
REPORTING_MANAGER_ESCALATION_HOURS=4
PROCESS_OWNER_ESCALATION_HOURS=8

# Trial Customer Configuration
TRIAL_DURATION_DAYS=30
TRIAL_REMINDER_DAYS=7,14,21,28
CONVERSION_CAMPAGN_ENABLED=true

# Backup Configuration
BACKUP_ENABLED=true
BACKUP_SCHEDULE=0 2 * * * # Daily at 2 AM
BACKUP_RETENTION_DAYS=30
```

10. Monitoring and Observability

10.1. Application Monitoring

10.1.1. Prometheus Metrics

```
// metrics.service.ts
import prometheus from 'prom-client';

// Create metrics registry
const register = new prometheus.Registry();

// Default metrics
prometheus.collectDefaultMetrics({ register });

// Custom application metrics
export const httpRequestDuration = new prometheus.Histogram({
  name: 'http_request_duration_seconds',
  help: 'Duration of HTTP requests in seconds',
  labelNames: ['method', 'route', 'status_code'],
  buckets: [0.1, 0.3, 0.5, 0.7, 1, 3, 5, 7, 10]
});

export const httpRequestTotal = new prometheus.Counter({
  name: 'http_requests_total',
  help: 'Total number of HTTP requests',
  labelNames: ['method', 'route', 'status_code']
});

export const orderProcessingDuration = new prometheus.Histogram({
  name: 'order_processing_duration_seconds',
  help: 'Duration of order processing in seconds',
  labelNames: ['order_type', 'fno_type'],
  buckets: [1, 5, 10, 30, 60, 300, 600, 1800, 3600]
});

export const fnoIntegrationErrors = new prometheus.Counter({
  name: 'fno_integration_errors_total',
  help: 'Total number of FNO integration errors',
  labelNames: ['fno_id', 'error_type']
});

export const escalationCount = new prometheus.Counter({
  name: 'escalations_total',
  help: 'Total number of escalations',
  labelNames: ['escalation_level', 'reason']
});

export const trialConversions = new prometheus.Counter({
  name: 'trial_conversions_total',
  help: 'Total number of trial conversions',
  labelNames: ['conversion_type']
});

// Register metrics
register.registerMetric(httpRequestDuration);
```

```
register.registerMetric(httpRequestTotal);
register.registerMetric(orderProcessingDuration);
register.registerMetric(fnoIntegrationErrors);
register.registerMetric(escalationCount);
register.registerMetric(trialConversions);

export { register };
```

10.1.2. Health Check Endpoints

```
// health.controller.ts
import { Request, Response } from 'express';
import { DatabaseService } from '@services/database.service';
import { RedisService } from '@services/redis.service';
import { KafkaService } from '@services/kafka.service';

export class HealthController {
  async healthCheck(req: Request, res: Response): Promise<void> {
    const health = {
      status: 'healthy',
      timestamp: new Date().toISOString(),
      uptime: process.uptime(),
      version: process.env.npm_package_version,
      environment: process.env.NODE_ENV,
      services: {
        database: await this.checkDatabase(),
        redis: await this.checkRedis(),
        kafka: await this.checkKafka()
      }
    };

    const isHealthy = Object.values(health.services).every(
      service => service.status === 'healthy'
    );

    if (!isHealthy) {
      health.status = 'unhealthy';
    }

    res.status(isHealthy ? 200 : 503).json(health);
  }

  private async checkDatabase(): Promise<{ status: string; responseTime?: number }> {
    try {
      const start = Date.now();
      await DatabaseService.query('SELECT 1');
      const responseTime = Date.now() - start;

      return { status: 'healthy', responseTime };
    } catch (error) {
      return { status: 'unhealthy' };
    }
  }

  private async checkRedis(): Promise<{ status: string; responseTime?: number }> {
    try {
      const start = Date.now();
      await RedisService.ping();
      const responseTime = Date.now() - start;

      return { status: 'healthy', responseTime };
    } catch (error) {
      return { status: 'unhealthy' };
    }
  }

  private async checkKafka(): Promise<{ status: string }> {
```

```
    try {  
      const isConnected = await KafkaService.isConnected();  
      return { status: isConnected ? 'healthy' : 'unhealthy' };  
    } catch (error) {  
      return { status: 'unhealthy' };  
    }  
  }  
}
```

10.2. Logging Configuration

10.2.1. Structured Logging

```
// logger.util.ts
import winston from 'winston';
import { ElasticsearchTransport } from 'winston-elasticsearch';

const logFormat = winston.format.combine(
  winston.format.timestamp(),
  winston.format.errors({ stack: true }),
  winston.format.json()
);

const transports: winston.transport[] = [
  new winston.transports.Console({
    format: winston.format.combine(
      winston.format.colorize(),
      winston.format.simple()
    )
  })
];

// Add file transport for production
if (process.env.NODE_ENV === 'production') {
  transports.push(
    new winston.transports.File({
      filename: 'logs/error.log',
      level: 'error',
      format: logFormat
    }),
    new winston.transports.File({
      filename: 'logs/combined.log',
      format: logFormat
    })
  );
}

// Add Elasticsearch transport if configured
if (process.env.ELASTICSEARCH_URL) {
  transports.push(
    new ElasticsearchTransport({
      level: 'info',
      clientOpts: {
        node: process.env.ELASTICSEARCH_URL
      },
      index: 'isp-oms-logs'
    })
  );
}

export const logger = winston.createLogger({
  level: process.env.LOG_LEVEL || 'info',
  format: logFormat,
  transports,
  defaultMeta: {
    service: 'isp-oms-backend',
    version: process.env.npm_package_version
  }
});
```

```
});  
  
// Audit logging  
export const auditLogger = winston.createLogger({  
  level: 'info',  
  format: logFormat,  
  transports: [  
    new winston.transports.File({  
      filename: 'logs/audit.log',  
      format: logFormat  
    })  
  ],  
  defaultMeta: {  
    type: 'audit',  
    service: 'isp-oms-backend'  
  }  
});
```

11. Security Implementation

11.1. Authentication Middleware

```
// auth.middleware.ts
import { Request, Response, NextFunction } from 'express';
import jwt from 'jsonwebtoken';
import { AuthenticationError, AuthorizationError } from '@utils/errors';
import { UserService } from '@services/user.service';
import { RedisService } from '@services/redis.service';

interface AuthenticatedRequest extends Request {
  user?: {
    id: string;
    email: string;
    role: string;
    permissions: string[];
  };
}

export const authenticate = async (
  req: AuthenticatedRequest,
  res: Response,
  next: NextFunction
): Promise<void> => {
  try {
    const authHeader = req.headers.authorization;

    if (!authHeader || !authHeader.startsWith('Bearer ')) {
      throw new AuthenticationError('No token provided');
    }

    const token = authHeader.substring(7);

    // Verify JWT token
    const decoded = jwt.verify(token, process.env.JWT_SECRET!) as any;

    // Check if token is blacklisted
    const isBlacklisted = await RedisService.get(`blacklist:${token}`);
    if (isBlacklisted) {
      throw new AuthenticationError('Token has been revoked');
    }

    // Get user details
    const user = await UserService.findById(decoded.userId);
    if (!user || !user.isActive) {
      throw new AuthenticationError('User not found or inactive');
    }

    // Attach user to request
    req.user = {
      id: user.id,
      email: user.email,
      role: user.role.name,
      permissions: user.role.permissions
    };

    // Update last activity
```

```

    await RedisService.set(
      `session:${decoded.userId}`,
      JSON.stringify({ lastActivity: Date.now() }),
      3600 // 1 hour
    );

    next();
  } catch (error) {
    if (error instanceof jwt.JsonWebTokenError) {
      next(new AuthenticationError('Invalid token'));
    } else {
      next(error);
    }
  }
};

export const authorize = (requiredPermissions: string[]) => {
  return (req: AuthenticatedRequest, res: Response, next: NextFunction): void
=> {
    if (!req.user) {
      return next(new AuthenticationError('User not authenticated'));
    }

    const hasPermission = requiredPermissions.every(permission =>
      req.user!.permissions.includes(permission)
    );

    if (!hasPermission) {
      return next(new AuthorizationError(
        `Insufficient permissions. Required: ${requiredPermissions.join(', ')}`
      ));
    }

    next();
  };
};

```

11.2. Input Validation Middleware

```
// validation.middleware.ts
import { Request, Response, NextFunction } from 'express';
import Joi from 'joi';
import { ValidationError } from '@utils/errors';

export const validate = (schema: Joi.ObjectSchema) => {
  return (req: Request, res: Response, next: NextFunction): void => {
    const { error, value } = schema.validate(req.body, {
      abortEarly: false,
      stripUnknown: true
    });

    if (error) {
      const errorMessage = error.details
        .map(detail => detail.message)
        .join(', ');

      return next(new ValidationError(errorMessage));
    }

    req.body = value;
    next();
  };
};

// Validation schemas
export const orderValidationSchema = Joi.object({
  customerId: Joi.string().uuid().required(),
  serviceType: Joi.string().valid('fiber', 'wireless', 'hybrid').required(),
  servicePackage: Joi.string().min(1).max(100).required(),
  installationAddress: Joi.object({
    street: Joi.string().min(1).max(200).required(),
    city: Joi.string().min(1).max(100).required(),
    province: Joi.string().min(1).max(100).required(),
    postalCode: Joi.string().pattern(/^\d{4}$/).required(),
    coordinates: Joi.object({
      latitude: Joi.number().min(-90).max(90),
      longitude: Joi.number().min(-180).max(180)
    }).optional()
  }).optional()
}).required(),
  priority: Joi.string().valid('low', 'normal', 'high', 'urgent').default('normal'),
  notes: Joi.string().max(1000).optional()
});

export const userValidationSchema = Joi.object({
  email: Joi.string().email().required(),
  firstName: Joi.string().min(1).max(100).required(),
  lastName: Joi.string().min(1).max(100).required(),
  phone: Joi.string().pattern(/^\+?[1-9]\d{1,14}$/).optional(),
  roleId: Joi.string().uuid().required(),
  reportingManagerId: Joi.string().uuid().optional()
});
```

12. Performance Optimization

12.1. Database Optimization

12.1.1. Connection Pooling

```
// database.config.ts
import { Pool } from 'pg';
import { MongoClient } from 'mongodb';
import Redis from 'ioredis';

// PostgreSQL connection pool
export const pgPool = new Pool({
  host: process.env.DB_HOST,
  port: parseInt(process.env.DB_PORT || '5432'),
  database: process.env.DB_NAME,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  min: 5,
  max: 20,
  idleTimeoutMillis: 30000,
  connectionTimeoutMillis: 2000,
  statement_timeout: 30000,
  query_timeout: 30000
});

// MongoDB connection
export const mongoClient = new MongoClient(process.env.MONGODB_URL!, {
  maxPoolSize: 10,
  minPoolSize: 2,
  maxIdleTimeMS: 30000,
  serverSelectionTimeoutMS: 5000,
  socketTimeoutMS: 45000
});

// Redis connection with clustering support
export const redisClient = new Redis.Cluster([
  {
    host: process.env.REDIS_HOST,
    port: parseInt(process.env.REDIS_PORT || '6379')
  }
], {
  redisOptions: {
    password: process.env.REDIS_PASSWORD,
    connectTimeout: 10000,
    commandTimeout: 5000,
    retryDelayOnFailover: 100,
    maxRetriesPerRequest: 3
  },
  enableOfflineQueue: false
});
```

12.1.2. Caching Strategy

```
// cache.service.ts
import { redisClient } from '@config/database.config';
import { logger } from '@utils/logger';

export class CacheService {
  private static readonly DEFAULT_TTL = 3600; // 1 hour

  static async get<T>(key: string): Promise<T | null> {
    try {
      const cached = await redisClient.get(key);
      return cached ? JSON.parse(cached) : null;
    } catch (error) {
      logger.error('Cache get error', { key, error });
      return null;
    }
  }

  static async set(key: string, value: any, ttl: number = this.DEFAULT_TTL):
  Promise<void> {
    try {
      await redisClient.setex(key, ttl, JSON.stringify(value));
    } catch (error) {
      logger.error('Cache set error', { key, error });
    }
  }

  static async del(key: string): Promise<void> {
    try {
      await redisClient.del(key);
    } catch (error) {
      logger.error('Cache delete error', { key, error });
    }
  }

  static async invalidatePattern(pattern: string): Promise<void> {
    try {
      const keys = await redisClient.keys(pattern);
      if (keys.length > 0) {
        await redisClient.del(...keys);
      }
    } catch (error) {
      logger.error('Cache pattern invalidation error', { pattern, error });
    }
  }

  // Cache decorators
  static cache(ttl: number = this.DEFAULT_TTL) {
    return function (target: any, propertyName: string, descriptor:
    PropertyDescriptor) {
      const method = descriptor.value;

      descriptor.value = async function (...args: any[]) {
        const cacheKey =
        `${target.constructor.name}:${propertyName}:${JSON.stringify(args)}`;

        // Try to get from cache
        const cached = await CacheService.get(cacheKey);
        if (cached !== null) {
          return cached;
        }
      };
    };
  }
}
```

```

    }

    // Execute method and cache result
    const result = await method.apply(this, args);
    await CacheService.set(cacheKey, result, ttl);

    return result;
  };
};
}
}

```

12.2. API Rate Limiting

```

// rate-limit.middleware.ts
import rateLimit from 'express-rate-limit';
import RedisStore from 'rate-limit-redis';
import { redisClient } from '@config/database.config';

// General API rate limiting
export const apiLimiter = rateLimit({
  store: new RedisStore({
    sendCommand: (...args: string[]) => redisClient.call(...args)
  }),
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // Limit each IP to 100 requests per windowMs
  message: {
    error: 'Too many requests from this IP, please try again later.'
  },
  standardHeaders: true,
  legacyHeaders: false
});

// Strict rate limiting for authentication endpoints
export const authLimiter = rateLimit({
  store: new RedisStore({
    sendCommand: (...args: string[]) => redisClient.call(...args)
  }),
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 5, // Limit each IP to 5 login attempts per windowMs
  message: {
    error: 'Too many login attempts from this IP, please try again later.'
  },
  skipSuccessfulRequests: true
});

// User-specific rate limiting
export const userLimiter = rateLimit({
  store: new RedisStore({
    sendCommand: (...args: string[]) => redisClient.call(...args)
  }),
  windowMs: 60 * 1000, // 1 minute
  max: 60, // Limit each user to 60 requests per minute
  keyGenerator: (req) => req.user?.id || req.ip,
  message: {
    error: 'Too many requests, please slow down.'
  }
});

```

13. Deployment Instructions

13.1. Replit Deployment Setup

13.1.1. Replit Configuration

```
// .replit
{
  "language": "nodejs",
  "run": "npm start",
  "entrypoint": "backend/src/index.ts",
  "hidden": [
    ".config",
    "package-lock.json"
  ],
  "audio": false,
  "supportedFeatures": {
    "packager": true,
    "interpreter": true,
    "lsp": true,
    "debugger": true
  },
  "packager": {
    "language": "nodejs",
    "features": {
      "packageSearch": true,
      "guessImports": true
    }
  },
  "debugger": {
    "support": true
  },
  "languages": {
    "nodejs": {
      "packager": {
        "language": "nodejs",
        "features": {
          "packageSearch": true,
          "guessImports": true,
          "enabledForHosting": false
        }
      }
    }
  }
}
```

13.1.2. Package.json Scripts

```
{
  "name": "isp-order-management-system",
  "version": "1.0.0",
  "description": "ISP Order Management System with Customer Onboarding",
  "main": "dist/index.js",
  "scripts": {
    "start": "node dist/index.js",
    "dev": "concurrently \"npm run dev:backend\" \"npm run dev:frontend\"",
    "dev:backend": "cd backend && npm run dev",
    "dev:frontend": "cd frontend && npm start",
    "build": "npm run build:backend && npm run build:frontend",
    "build:backend": "cd backend && npm run build",
    "build:frontend": "cd frontend && npm run build",
    "test": "npm run test:backend && npm run test:frontend",
    "test:backend": "cd backend && npm test",
    "test:frontend": "cd frontend && npm test",
    "migrate": "cd backend && npm run migrate",
    "seed": "cd backend && npm run seed",
    "setup": "npm run install:all && npm run migrate && npm run seed",
    "install:all": "npm install && cd backend && npm install && cd ../frontend
&& npm install",
    "docker:build": "docker-compose build",
    "docker:up": "docker-compose up -d",
    "docker:down": "docker-compose down",
    "docker:logs": "docker-compose logs -f"
  },
  "dependencies": {
    "concurrently": "^7.6.0"
  },
  "devDependencies": {
    "@types/node": "^18.0.0",
    "typescript": "^4.9.0"
  }
}
```

13.2. Environment Setup Script

```
#!/bin/bash
# setup.sh - Initial setup script for Replit

echo "🚀 Setting up ISP Order Management System..."

# Install dependencies
echo "📦 Installing dependencies..."
npm run install:all

# Setup environment files
echo "⚙️ Setting up environment configuration..."
if [ ! -f backend/.env ]; then
    cp backend/.env.example backend/.env
    echo "✅ Created backend/.env from example"
fi

if [ ! -f frontend/.env ]; then
    cp frontend/.env.example frontend/.env
    echo "✅ Created frontend/.env from example"
fi

# Build applications
echo "🔨 Building applications..."
npm run build

# Setup database
echo "🗄️ Setting up database..."
npm run migrate
npm run seed

echo "✅ Setup complete! Run 'npm run dev' to start development servers."
echo "📖 Check README.md for additional configuration steps."
```

13.3. Production Deployment Checklist

13.3.1. Pre-deployment Checklist

Production Deployment Checklist

Security

- [] All environment variables are properly configured
- [] JWT secrets are strong and unique
- [] Database passwords are secure
- [] API keys are properly encrypted
- [] HTTPS is enabled with valid SSL certificates
- [] CORS is properly configured
- [] Rate limiting is enabled
- [] Input validation is implemented

Database

- [] Database migrations are up to date
- [] Database backups are configured
- [] Database connection pooling is optimized
- [] Indexes are created for performance
- [] Database monitoring is enabled

Performance

- [] Caching is properly configured
- [] CDN is set up for static assets
- [] Image optimization is enabled
- [] Gzip compression is enabled
- [] Database queries are optimized
- [] API response times are acceptable

Monitoring

- [] Application monitoring is configured
- [] Error tracking is enabled
- [] Log aggregation is set up
- [] Health checks are implemented
- [] Alerting is configured
- [] Performance metrics are tracked

Testing

- [] All unit tests pass
- [] Integration tests pass
- [] End-to-end tests pass
- [] Load testing is completed
- [] Security testing is completed

Documentation

- [] API documentation is up to date
- [] Deployment documentation is complete
- [] User documentation is available
- [] Troubleshooting guide is prepared

14. Maintenance and Support

14.1. Backup Strategy

```
// backup.service.ts
import { exec } from 'child_process';
import { promisify } from 'util';
import { logger } from '@utils/logger';

const execAsync = promisify(exec);

export class BackupService {
  static async createDatabaseBackup(): Promise<string> {
    try {
      const timestamp = new Date().toISOString().replace(/[:.]/g, '-');
      const backupFile = `backup-${timestamp}.sql`;

      const command = `pg_dump ${process.env.DATABASE_URL} >
backups/${backupFile}`;
      await execAsync(command);

      logger.info('Database backup created', { backupFile });
      return backupFile;
    } catch (error) {
      logger.error('Database backup failed', { error });
      throw error;
    }
  }

  static async createMongoBackup(): Promise<string> {
    try {
      const timestamp = new Date().toISOString().replace(/[:.]/g, '-');
      const backupDir = `mongo-backup-${timestamp}`;

      const command = `mongodump --uri="${process.env.MONGODB_URL}" --
out=backups/${backupDir}`;
      await execAsync(command);

      logger.info('MongoDB backup created', { backupDir });
      return backupDir;
    } catch (error) {
      logger.error('MongoDB backup failed', { error });
      throw error;
    }
  }

  static async scheduleBackups(): Promise<void> {
    // Schedule daily backups at 2 AM
    const cron = require('node-cron');

    cron.schedule('0 2 * * *', async () => {
      try {
        await this.createDatabaseBackup();
        await this.createMongoBackup();
        await this.cleanupOldBackups();
      } catch (error) {
        logger.error('Scheduled backup failed', { error });
      }
    });
  }
}
```

```
    });  
  }  
  
  private static async cleanupOldBackups(): Promise<void> {  
    const retentionDays = parseInt(process.env.BACKUP_RETENTION_DAYS || '30');  
    const cutoffDate = new Date();  
    cutoffDate.setDate(cutoffDate.getDate() - retentionDays);  
  
    // Implementation for cleaning up old backup files  
    // This would typically involve checking file dates and removing old files  
  }  
}
```

14.2. System Maintenance

```
// maintenance.service.ts
export class MaintenanceService {
  static async performDailyMaintenance(): Promise<void> {
    logger.info('Starting daily maintenance tasks');

    try {
      // Clean up expired sessions
      await this.cleanupExpiredSessions();

      // Archive old audit logs
      await this.archiveOldAuditLogs();

      // Update system statistics
      await this.updateSystemStatistics();

      // Check system health
      await this.performHealthChecks();

      logger.info('Daily maintenance completed successfully');
    } catch (error) {
      logger.error('Daily maintenance failed', { error });
    }
  }

  private static async cleanupExpiredSessions(): Promise<void> {
    // Remove expired sessions from Redis
    const expiredSessions = await redisClient.keys('session:*');
    // Implementation for cleaning expired sessions
  }

  private static async archiveOldAuditLogs(): Promise<void> {
    // Archive audit logs older than 1 year
    const archiveDate = new Date();
    archiveDate.setFullYear(archiveDate.getFullYear() - 1);

    // Implementation for archiving old audit logs
  }

  private static async updateSystemStatistics(): Promise<void> {
    // Update daily/weekly/monthly statistics
    // Implementation for updating system statistics
  }

  private static async performHealthChecks(): Promise<void> {
    // Perform comprehensive health checks
    // Implementation for health checks
  }
}
```

This comprehensive technical specification document provides all the necessary information for developers to build the ISP Order Management System on Replit. The document covers architecture, database design, API specifications, security requirements, deployment instructions, and maintenance procedures. Each section

includes detailed code examples and configuration files to ensure successful implementation.